

upGrad

From data to a deployable churn signal

A compact Applied AI and AWS teaching pattern library

Session map

01 Frame the decision

Connect a business decision, a target and a point-in-time data contract.

02 Build the baseline

Use reproducible Python, SQL and an architecture with traceable responsibilities.

03 Close the loop

Practise judgment and end with an operational takeaway.

04 TeXScience diagrams

Use editable scientific plots and diagrams without a remote rendering dependency.

Each topic can be compiled alone while this session file assembles the complete learner deck.

TOPIC 04

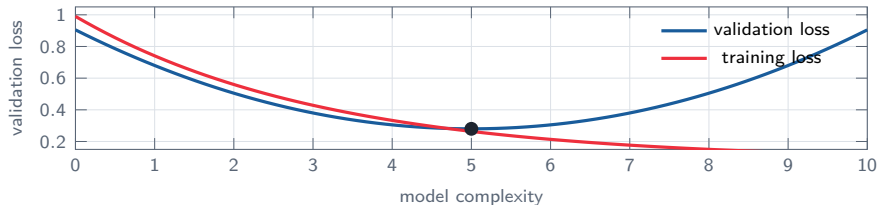
TeXScience diagrams

Use editable scientific plots and diagrams without a remote rendering dependency.

[Sample framework](#) / [Living documentation](#)

Scientific plot: expose the trade-off, not just the number

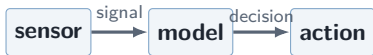
Illustrative validation curves



The plot is PGFPlots source: change the functions, units or labels in version control rather than redrawing an image.

Scientific diagram: make the system inspectable

Measurement path



Editable TikZ node-and-flow diagram.

TeXScience support

- `pgfplots` for charts
- `siunitx` for values such as 12.5 ms
- `circuitikz`, `chemfig` and `tikz-3dplot` for domain diagrams
- no shell escape or web renderer required

Agenda

1. Frame the learner's decision and the data needed to support it.
2. Build a reproducible baseline in Python and SQL.
3. Move from notebook insight to an AWS-ready delivery path.
4. Practise the judgment calls that make a model useful.

01

Start with a decision

A useful model changes a decision; accuracy is only one part of the story.

A normal content slide: define the learning path

Business question

Which subscribers should receive a retention offer this week, given a limited outreach budget?

- Decision owner: customer-success lead
- Unit: one active subscriber
- Outcome horizon: churn in the next 30 days

Example – A measurable first hypothesis

Recent support friction, low tenure and a falling engagement trend may raise churn risk.

- Train on observations available before outreach.
- Compare against a no-model targeting baseline.
- Review false positives with the operations team.

Dense teaching slide: the minimum viable data contract

Field	Available at scoring time?	Why it belongs (or does not)
tenure_months	Yes	Account age recorded on the scoring date.
support_tickets_30d	Yes	Recent friction; define timezone and ticket-status rules.
cancelled_at	No	Future outcome: using it would leak the answer.
plan	Yes	Categorical context; preserve an unknown route.
offer_accepted	Depends	Include only if offer assignment preceded the outcome.

Watch for – Time leakage is a product bug

If a column is created after the decision, a high offline score can be misleading rather than useful.

Learning objectives

By the end of this session, learners can

- translate a retention decision into a supervised-learning target;
- build a preprocessing-and-model pipeline without feature leakage;
- write a query that produces a point-in-time feature table; and
- explain how a model artefact reaches a monitored endpoint.

A concept and its boundary

Definition – Point-in-time correctness

Every feature value must be computed only from information available on or before the timestamp at which the prediction is made.

Example – Correct

Score on 1 July using support tickets opened through 30 June.

Watch for – Incorrect

Score on 1 July using a “days since cancellation” field populated on 15 July.

02

Make the baseline reproducible

A small, explicit pipeline is easier to debug, explain and deploy.

Python: highlight the lines that matter

```
1 features = ["monthly_spend", "tenure_months", "support_tickets", "plan"]
2 preprocess = ColumnTransformer([
3     ("numeric", StandardScaler(), features[:3]),
4     ("category", OneHotEncoder(handle_unknown="ignore"), ["plan"]),
5 ])
6 model = Pipeline([
7     ("prepare", preprocess),
8     ("classifier", LogisticRegression(max_iter=1_000)),
9 ])
10 model.fit(train[features], train["churned"])
11 probability = model.predict_proba(customer[features])[:, 1]
12
```

Highlighted lines are selected with `{7,10,11}`; use `numbers=left` when learners need to discuss a line precisely.

Code plus an adjacent teaching annotation

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import OneHotEncoder, StandardScaler
5
6 features = ["monthly_spend", "tenure_months", "support_tickets", "
    plan"]
7 preprocess = ColumnTransformer([
8     ("numeric", StandardScaler(), features[:3]),
9     ("category", OneHotEncoder(handle_unknown="ignore"), ["plan"]),
10 ])
11 model = Pipeline([
12     ("prepare", preprocess),
13     ("classifier", LogisticRegression(max_iter=1_000)),
14 ])
15 model.fit(train[features], train["churned"])
16 churn_probability = model.predict_proba(customer[features])[:, 1]
```

Lines 11, 13–14: train and infer

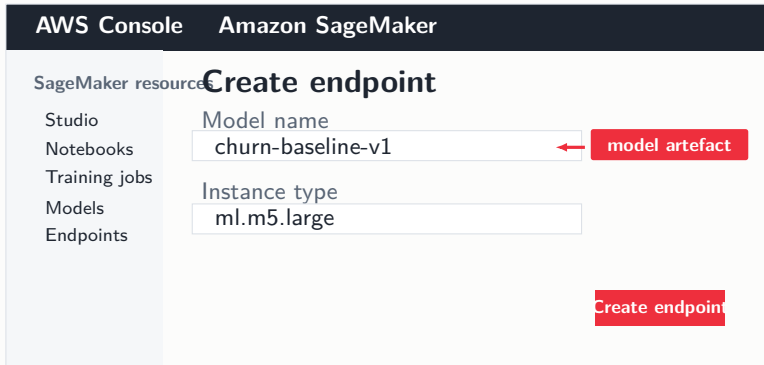
The pipeline keeps preprocessing and the classifier together. **Line 13** learns parameters only from the training split; **line 14** produces a probability, not a retention decision. The business threshold is a separate policy choice.

SQL: create a stable feature snapshot

```
1 WITH feature_snapshot AS (  
2     SELECT  
3         customer_id,  
4         DATE '2026-06-30' AS scored_on,  
5         COUNTIF(created_at >= DATE_SUB(DATE '2026-06-30', INTERVAL 30 DAY))  
6             AS support_tickets_30d,  
7         MAX(plan) AS plan  
8     FROM analytics.support_events  
9     WHERE created_at < TIMESTAMP '2026-07-01 00:00:00 UTC'  
10    GROUP BY customer_id  
11 )  
12 SELECT * FROM feature_snapshot;  
13
```

The exclusive upper bound is intentional: the snapshot must not read events that arrive after scoring begins.

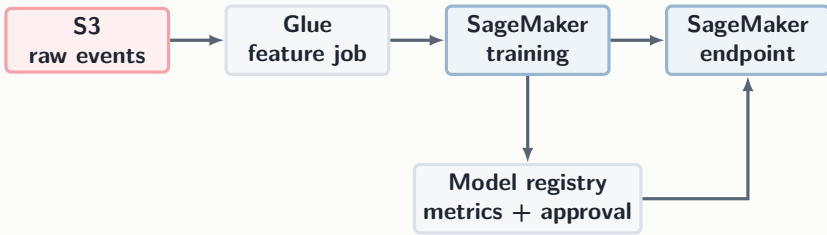
AWS Console screenshot placeholder



A self-contained placeholder. Replace this TikZ mock with

`\begin{AnnotatedScreenshot}{assets/aws-endpoint.png}{...}` and add percentage-positioned `\ScreenshotCallouts`.

Architecture: separate responsibilities, preserve traceability



Key takeaway – A deployment is a chain of evidence

The endpoint should be traceable to a versioned dataset, training run, evaluation result and approval decision.

Compare options against the decision constraint

Option	Strength	Risk	Use when
Logistic regression	Fast, inspectable baseline	Misses non-linear interactions	You need a transparent benchmark.
Gradient boosting	Strong tabular accuracy	More tuning and monitoring	Incremental lift justifies complexity.
Rules only	Easy to audit	Brittle thresholds, limited coverage	Policy rules are explicit.

A question for the room

Question – Which threshold should trigger an outreach offer?

A probability of 0.62 is not automatically “high enough”. What are the cost of a false positive, the value of a retained subscriber, and the daily outreach capacity?

Discuss

- What action follows a positive prediction?
- Who is harmed by an unnecessary offer?

Measure

- precision among contacted subscribers
- incremental retention against a control group

Exercise: repair the evaluation design

Try it – Ten minutes, pairs

A team trained on January–June data and randomly split rows into train and test. They report 92% AUC, then score July subscribers. Identify two ways this design can overstate production performance. Redesign the split in one sentence.

Example – Facilitator debrief

A time-based split better simulates deployment. Check that the label horizon ends before the evaluation period and that one customer cannot appear on both sides through repeated snapshots.

Case study: the model is right, the intervention is wrong

Case study – Retention offer fatigue

A model identifies high-risk annual-plan subscribers correctly. The team sends a discount to every one of them; renewal rises, but margin falls because many would have renewed without an offer.

- Prediction answers: *who is likely to churn?*
- Uplift or experiment design answers: *who changes behaviour because of an offer?*
- Operational policy answers: *which action is viable and fair?*

03

Close the loop

A slide deck should end in a decision, a practice task and a durable mental model.

Key takeaway

Key takeaway – Build the decision boundary before the model boundary

Define the decision timestamp, the intervention and the costs first. Then make features point-in-time correct, train a reproducible pipeline, and evaluate against the future workflow it will serve.

Recap

1. A supervised-learning target is a decision framed at a timestamp.
2. Point-in-time feature definitions prevent leakage.
3. Pipelines keep preprocessing and model behaviour reproducible.
4. An AWS endpoint needs lineage, monitoring and a human policy around it.

Next: turn the baseline into an experiment.

Bring one proposed feature and the timestamp at which it becomes available.

upGrad

References

- AWS, *Amazon SageMaker Developer Guide*: model deployment, registry and endpoint concepts.
- scikit-learn, *Pipeline* and *ColumnTransformer* API documentation.